

实验三 父子进程间数据段的关系

1. 实验目的

掌握并进一步认识并发进程的实质，理解父子进程间数据段的关系。

2. 相关知识

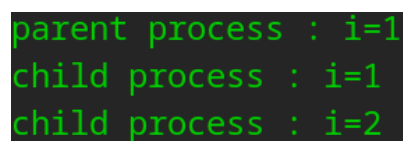
子进程继承父进程的程序、数据等资源；子进程刚创建时，复制父进程的数据段，与父进程的数据是一样的；在接下来的代码中，父子进程可以修改各自的数据而互不影响。

3. 实验程序

【程序 3_3】用 `fork()` 系统调用来创建一个子进程，并在子进程中对变量赋值，观察父进程和子进程中变量的值。

```
1. #include <stdio.h>
2. #include <unistd.h>
3. #include <stdlib.h>
4. void main(){
5.     int i = 1;
6.     if(fork() == 0){
7.         printf("child process:i=%d\n",i);
8.         i = 2;
9.         printf("child process:i=%d\n",i);
10.        exit(0);
11.    }
12.    printf("parent process:i=%d\n",i);
13. }
```

【程序运行结果截图及运行说明】



```
parent process : i=1
child process  : i=1
child process  : i=2
```

程序思考：程序的输出一定是上图的方式吗？输出的三行顺序是否有可能改变为什么？

【程序 3_4】将上述【程序 3_3】中子进程的 `exit(0)` 去掉，并将父进程的输出语句做修改，以便能够看出输出对应的是哪个进程的输出语句、是哪个进程中的变量。

```
1. #include <stdio.h>
2. #include <unistd.h>
3. #include <stdlib.h>
4. void main(){
5.     int i = 1;
6.     if(fork() == 0){
7.         printf("child process:i=%d\n",i);
```

```

8.         i = 2;
9.         printf("child process:i=%d\n",i);
10.    }
11.    printf("child process pid is %d, parent process pid is %d,i=%d\n",getpid
        (),getppid(),i);
12. }

```

【程序运行结果截图及运行说明】

```

child process pid is : 13987, parent process pid is : 2671, i = 1
child process : i=1
child process : i=2
child process pid is : 13988, parent process pid is : 1, i = 2
child process pid is : 14041, parent process pid is : 2671, i = 1
child process : i=1
child process : i=2
child process pid is : 14042, parent process pid is : 14041, i = 2

```

程序思考：

1. 根据程序代码分析各行输出分别是执行了程序中的哪个 printf 语句；根据进程 pid 分析各行输出分别对应哪个进程中的 printf 语句；两种分析的结果应该是一致的。
2. 当程序的运行结果中出现上面的第一种情况，怎样解释呢？

child process pid is : 13988, parent process pid is: 1, i = 2

提示：因为父进程比子进程先结束，此时，子进程相当于孤儿进程，该子进程应该由 init 进程领养，所以，子进程的父进程的 pid 此时变为 1。

【程序 3_5】将【程序 3_4】略做变换，增加 else，并简化最后一行输出。运行程序，分析其结果与【程序 3_4】的不同。

```

1. #include <stdio.h>
2. #include <unistd.h>
3. #include <stdlib.h>
4. void main(){
5.     int i = 1;
6.     if(fork() == 0){
7.         printf("child process pid is: %d\n",getpid());
8.         printf("child process:i = %d\n",i);
9.         i = 2;
10.        printf("child process:i = %d\n",i);
11.    }
12.    else
13.        printf("parent process pid is %d: i = %d\n",getpid(),i);
14. }

```

【程序运行结果截图及运行说明】

```
parent process pid is 7714: i = 1
child process pid is: 7715
child process:i = 1
child process:i = 2
```

【程序 3_6】定义一个全局变量 globa，并增加两个语句标号 k1 和 k2，试运行。

```
1. #include <stdio.h>
2. #include <unistd.h>
3. #include <stdlib.h>
4. int globa = 4;
5. void main(){
6.     pid_t pid;
7.     int vari = 5;
8.     printf("before fork.\n");
9.
10.    if((pid = fork()) < 0){
11.        printf("fork error.\n");
12.        exit(0);
13.    }
14.    else if ( pid == 0){
15.        globa ++;
16.        vari --;
17.        printf("Child changed the vari and globa.\n");
18.    }
19.    else
20.        printf("Parent did not chang the vari and globa.\n");
21.
22.    k1: printf("globa = %d, vari = %d\n",globa,vari);
23.    k2: exit(0);
24. }
```

【程序运行结果截图及运行说明】

```
before fork.
Parent did not chang the vari and globa.
globa = 4, vari = 5
Child changed the vari and globa.
globa = 5, vari = 4
```

- (1) 运行程序，分析其结果；
- (2) 修改程序，以便能够看出标号为 K1 的语句属于哪个进程；
- (3) 在最后一个 printf 中，当加上：getpid() 和 getppid() 时，查看父子进程号并分析运行结果。